As a small example, let's use these parameters on the `destroy` abilities we have defined, as shown in the following code snippet:

```
# We can write
can :destroy, Post, user_id: user.id
# as a new rule that looks similar
can :destroy, Post, ['user_id = ?', user.id]
```



Note that the new rule is not quite the same as the old one; the SQL query will only help build queries using the CanCan model scope `Model#accessible_by(ability, action)`; it will not work if/when you apply the rule on your views.

To make it work on views, implement a block that looks equivalent to the SQL. CanCan will use the block when you apply checks on the views. So, the correct block-based definition for the earlier rule is as follows:

```
can :destroy, Post, ['user_id = ?', user.id] do |post|
  post.user_id == user.id
end
```

The block is evaluated when an instance of post is passed for checking. You should not do something, as shown in the following snippet:

```
# Admin can do anything
can :manage, :all do |post|
  user.admin?
end
```

In some cases, for instance, when you try to check a user's ability to create a new post using `can?(:create, Post)`, the block above will not get called. Such behavior potentially breaks your application. Instead, the correct way is as follows:

```
if user.admin?
  can :manage, :all # An administrator can do anything
else
  # Regular user
end
```

Now that we know how to use SQL conditions and blocks to implement abilities, let's try to use them for abilities that require SQL subselects, as shown in the following sections.

www.it-ebooks.info